

**Московский авиационный институт
(национальный исследовательский университет)**

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторные работы по курсу “Информационный поиск”

Студент: Л. А. Филенков
Преподаватель: А. А. Кухтичев
Группа: М8О-407Б-22
Дата: 27.12.2025
Оценка:
Подпись:

Лабораторная работа №1 “Добыча корпуса документов”

Необходимо проанализировать корпус документов, который будет использован при выполнении остальных лабораторных работ:

- Скачать примеры документов к себе на компьютер. В отчёте нужно указать источник данных. Источников в итоговом индексе должно быть не менее двух;
- Ознакомиться с ним, изучить его характеристики. Из чего состоит текст? Есть ли дополнительная мета-информация? Если разметка текста, какая она?
- Выделить текст.
- Найти существующие поисковики, которые уже можно использовать для поиска по выбранному набору документов (встроенный поиск Википедии, поиск Google с использованием ограничений на URL или на сайт). Если такого поиска найти невозможно, то использовать корпус для выполнения лабораторных работ нельзя!
- Привести несколько примеров запросов к существующим поисковикам, указать недостатки в полученной поисковой выдаче.

В результатах работы должна быть указаны статистическая информация о корпусе:

- Размер примеров «сырых» документов.
- Количество документов.
- Размер текста, выделенного из «сырых» данных.
- Средний размер документа, средний объём текста в документе.

Цель работы

Создание фундаментальной базы данных для информационного поиска — текстового корпуса. Задание требует разработки программного обеспечения для автоматического сбора веб-страниц с соблюдением ряда требований: объем корпуса не менее 100 000 документов, сохранение исходного HTML и выделенного текста, а также метаданных (URL, дата). Данные должны быть получены из независимых источников для обеспечения репрезентативности.

Источники данных

Для формирования корпуса были выбраны региональные новостные порталы и сайты административных округов, отличающиеся глубокой архивацией материалов и стабильной структурой HTML.

Использовались следующие 7 источников:

1. Городские новости (Ярославль) — city-news.ru
2. Ковровские вести (Ковров) — xn--b1aaalbpdjclbbxpfp.xn--p1ai
3. Интернет-газета района Старое Крюково (Зеленоград) — staroekrukovo.ru
4. Наше Силино (Зеленоград) — nashesilino.ru
5. VladNews (Владивосток) — vladnews.ru
6. Newsroom24 (Нижний Новгород) — newsroom24.ru
7. Администрация Серпухова — serpuhov.ru

Описание программы

Программа реализована на языке Python с использованием асинхронного подхода (библиотека aiohttp) для обеспечения высокой производительности при сетевых операциях. Архитектура системы построена по модульному принципу и включает компоненты для конфигурации, сетевого взаимодействия, парсинга и хранения данных.

Основным конфигурационным элементом является YAML-файл, управляющий правилами обхода. Для каждого источника создаются объекты правил CompiledSourceRules, содержащие скомпилированные регулярные выражения. Параметр allow_regex разрешает навигацию по страницам списков, save_regex определяет целевые страницы статей, подлежащие сохранению, а deny_regex блокирует загрузку технических и

бинарных файлов.

Ядром системы выступает класс `Crawler`, который управляет очередью ссылок и состоянием обхода. При старте метод `_resume_from_manifest` считывает файл манифеста, восстанавливая множество уже посещенных URL и хешей, что позволяет продолжать сбор данных после перезапуска без дублирования запросов. Метод `run` запускает пул рабочих корутин, количество которых регулируется параметром `concurrency`.

Сетевой модуль использует `aiohttp.ClientSession` для выполнения HTTP-запросов, обрабатывая ошибки и коды состояния (429, 503) с помощью механизма экспоненциальной задержки. Модуль парсинга `extract_text_from_html` на базе `BeautifulSoup` очищает HTML-код от элементов скриптов, стилей и навигации, извлекая полезный текстовый контент.

Для сохранения данных используется файловая структура. Каждому уникальному документу присваивается ID. Исходный код сохраняется в директорию `raw_html`, очищенный текст — в `text`, а метаданные в формате JSON — в `meta`. Дедупликация реализована через вычисление SHA-256 хеша от очищенного текста: документы с совпадающим хешем отбрасываются. Все операции фиксируются в глобальном реестре `manifest.csv`.

Результаты

Использование асинхронной архитектуры позволило достичь скорости обработки, достаточной для скачивания 100 000 документов за приемлемое время. Механизм фильтрации по регулярным выражениям обеспечил чистоту корпуса, исключив попадание в выборку нерелевантных страниц.

Статистика собранного корпуса:

Documents: 105 925

RAW sum MB: 5772.32

RAW avg KB: 54.41

TEXT sum MB: 969.64

TEXT avg KB: 9.37

Выводы

В результате выполнения лабораторной работы был реализован высокопроизводительный асинхронный краулер для сбора текстового корпуса. Сложности программирования были связаны с настройкой регулярных выражений для корректной фильтрации URL на различных сайтах, а также с обработкой кодировок и сетевых ошибок при массовом скачивании.

Полученный корпус документов полностью готов к использованию в следующих лабораторных работах: сохраненные файлы станут основой для токенизации и построения индекса. Реализованная схема хранения обеспечивает эффективный прямой доступ к документам по их идентификатору.

Лабораторная работа №2 “Поисковый робот”

Необходимо написать парсер на любом языке программирования.

- Написать поисковый робот — компоненты обкачки документов, используя любой язык программирования;
- Единственным аргументом поисковому роботу подаётся путь до yaml-конфига, содержащий:
 - Данные для базы данных в секции db;
 - Данные для робота в секции logic: задержка между обкачкой страницы;
 - Любые другие данные, необходимые для реализации логики поискового робота.
- Сохранять в базе данных (например, MongoDB) документы со следующими полями:
 - url, нормализованный;
 - «сырой» html-текст документа;
 - название источника;
 - Дата обкачки документа в формате Unix time stamp.
- Поисковый робот можно остановить в любой момент и при повторном запуске робот должен начать с того документа, с которого он остановился;
- Периодически он должен уметь переобкачивать документы, которые уже есть в базе, но только в том случае, если они изменились.

Цель работы

Создание автоматизированного поискового робота для непрерывного сбора и обновления информации из веб-источников. Задание требует разработки программы, которая управляется конфигурационным файлом, обеспечивает сохранение данных в базу данных (MongoDB) и поддерживает функциональность остановки/возобновления работы. Важным требованием является реализация логики повторного скачивания (recrawl): робот должен периодически проверять уже известные документы и обновлять их в базе только в случае изменения контента.

Источники данных

В работе использовались те же источники, что и в первой лабораторной работе, для обеспечения преемственности данных и тестирования механизма обновлений:

1. Городские новости (Ярославль) — city-news.ru
2. Ковровские вести (Ковров) — xn--b1aaalbpdjclbbxpfpxn--p1ai
3. Интернет-газета района Старое Крюково (Зеленоград) — staroekrukovo.ru
4. Наше Силино (Зеленоград) — nashesilino.ru
5. VladNews (Владивосток) — vladnews.ru
6. Newsroom24 (Нижний Новгород) — newsroom24.ru
7. Администрация Серпухова — serpuhov.ru

Описание программы

Программа представляет собой развитие архитектуры краулера из первой лабораторной работы, дополненное новыми компонентами для работы с базой данных и управлением состоянием обхода. Реализация выполнена на языке Python с использованием асинхронных библиотек aiohttp, pymongo.

Центральным элементом системы является класс Robot, который инициализируется через YAML-конфигурацию. В отличие от первой работы, где состояние хранилось в файловой системе, здесь внедрена абстракция MongoFrontier для управления очередью ссылок. Фронтир хранит URL, статус скачивания, время следующей проверки и глубину в коллекции MongoDB. Это позволяет роботу корректно останавливаться и возобновлять работу с прерванного места, просто считывая незавершенные задачи из базы.

Для хранения документов реализован класс `MongoStorage`. Он использует операцию `upsert` (`update` or `insert`) для атомарного обновления данных. При попытке сохранить документ программа вычисляет хеш (SHA-256) его содержимого. Если документ с таким нормализованным URL уже существует в базе, происходит сверка хешей. Обновление записи выполняется только при несовпадении хешей, что минимизирует избыточную нагрузку на БД и сеть. Схема документа в MongoDB включает поля: `url_norm`, `source_id`, `fetch_status`, `content_hash` и `crawled_at`.

Логика повторного обхода (`recrawl`) реализована через поле `next_fetch_at` во фронтире. Если документ успешно скачан, ему выставляется тайм-аут, после чего он снова становится доступным для выбора воркерами. При ошибках скачивания применяется стратегия экспоненциальной задержки.

Результаты

Реализованный поисковый робот успешно интегрирован с базой данных MongoDB. Тестирование показало корректную работу механизмов остановки и возобновления (`resume`): после перезапуска процесса сбор данных продолжался без потерь. Функция `recrawl` подтвердила свою работоспособность: при изменении контента на тестовых страницах робот обновлял соответствующие записи в базе.

Статистика базы данных после миграции и докачки:

Documents: 108640

Per source: {'city_news_yaroslavl': 31222, 'kovrovskie_vesti_kovrov': 26543, 'newsroom24_nnov': 6680, 'serpuhov_admin': 11541, 'vladnews_vladivostok': 9123, 'zelenograd_silino': 3907, 'zelenograd_staroekrukovo': 19624}

Структура одного элемента в documents:

```
{
  _id: ObjectId('6944478715a57e30a8da55fe'),
  url_norm: 'https://city-news.ru/news/',
  bytes_len: 55313,
  checked_at: 1765883003,
  content_hash: '5bb4df821b08b2bd6e7713abfba9ad1a95fd6b7f6a767b228220f369822c48f6',
  content_type: 'text/html',
  crawled_at: 1765883003,
  encoding: 'utf-8',
  first_seen_at: 1765883003,
  source_id: 'city_news_yaroslavl',
  source_name: 'Городские новости (Ярославль)',
  status: 200,
  url: 'https://city-news.ru/news/'
}
```

Структура одного элемента в frontier:

```
{
  _id: ObjectId('694453cf15a57e30a8dda68e'),
  url_norm: 'https://city-news.ru/news/',
  added_at: 1765883003,
  depth: 0,
  next_fetch_at: 1766690382,
  source_id: 'city_news_yaroslavl',
  status: 'pending',
  updated_at: 1765883003,
}
```

```
url: 'https://city-news.ru/news/'  
}
```

Найденные документы являлись служебными страницами, не содержащими текста для извлечения, поэтому число документов для дальнейшей работы осталось равным числу из ЛР1.

Выводы

В результате выполнения лабораторной работы была создана полноценная система управления жизненным циклом веб-документов. Переход от файлового хранения к базе данных MongoDB позволил реализовать сложные сценарии обновления данных и управления очередью скачивания, которые труднодостижимы на файловой системе.

Сложности программирования были связаны с реализацией эффективного фронта на базе MongoDB и синхронизацией асинхронных воркеров. Полученная система является надежным бэкендом для поисковой машины, обеспечивая постоянную актуальность за счет регулярного переобхода источников.

Лабораторные работы №3, №5 “Токенизация” и “Закон Ципфа”

Нужно реализовать процесс разбиения текстов документов на токены, который потом будет использоваться при индексации. Для этого потребуется выработать правила, по которым текст делится на токены. Необходимо описать их в отчёте, указать достоинства и недостатки выбранного метода. Привести примеры токенов, которые были выделены неудачно, объяснить, как можно было бы поправить правила, чтобы исправить найденные проблемы.

В результатах выполнения работы нужно указать следующие статистические данные:

- Количество токенов.
- Среднюю длину токена.

Кроме того, нужно привести время выполнения программы, указать зависимость времени от объёма входных данных. Указать скорость токенизации в расчёте на килобайт входного текста. Является ли эта скорость оптимальной? Как её можно ускорить?

Для своего корпуса необходимо построить график распределения терминов по частотностям в логарифмической шкале, наложить на этот график закон Ципфа. Объяснить причины расхождения.

В качестве дополнительного задания, можно (но не обязательно) подобрать константы для закона Мандельброта, наложить полученный график на график распределения терминов по частотностям. Привести выбранные константы.

Цель работы

Задача состоит в разбиении собранных ранее документов на отдельные токены, нормализации их к нижнему регистру и подсчете статистик встречаемости. Результатом работы является коллекция терминов с их частотами, а также проверка соответствия распределения слов закону Ципфа. Важным условием является создание программного модуля, способного обработать весь собранный объем данных.

Описание программы

Программная реализация выполнена на языке C без STL в соответствии с требованиями. Приложение взаимодействует с базой данных MongoDB через драйвер mongoc, извлекая документы и сохраняя результаты анализа. Ключевым элементом программы является модуль токенизации. Алгоритм проходит по байтовому массиву текста в кодировке UTF-8, идентифицируя последовательности символов, относящихся к категории букв или цифр. Для классификации символов Unicode используется таблица свойств кодовых точек, позволяющая корректно обрабатывать как латиницу, так и кириллицу. Все выделенные токены приводятся к нижнему регистру путем смещения кодов соответствующих символов. Знаки препинания, пробелы и управляющие символы игнорируются.

Для подсчета частот используется собственная реализация хеш-таблицы с открытой адресацией. Ключом выступает строковое представление токена, а значением — счетчик его вхождений в корпус. Такая структура данных обеспечивает константное время доступа при вставке и обновлении частот, что критично при словаре в сотни тысяч уникальных слов.

Программа работает в поточном режиме:

1. Читает записи из манифеста или базы данных.
2. Загружает текстовый файл соответствующего документа.
3. Разбивает текст на токены, обновляя глобальную хеш-таблицу частот.
4. Выгружает итоговую статистику в коллекцию term_stats базы данных и в CSV-файл для построения графиков.

Дополнительно реализован скрипт на Python для визуализации результатов. Он строит график рангового распределения частот в логарифмическом масштабе, позволяя наглядно оценить соответствие эмпирических данных теоретическому закону Ципфа.

Результаты

Программа успешно обработала весь корпус документов.

Статистические показатели корпуса:

DONE

docs=105925

tokens=66046912

unique_terms=445976

avg_token_len=7.122

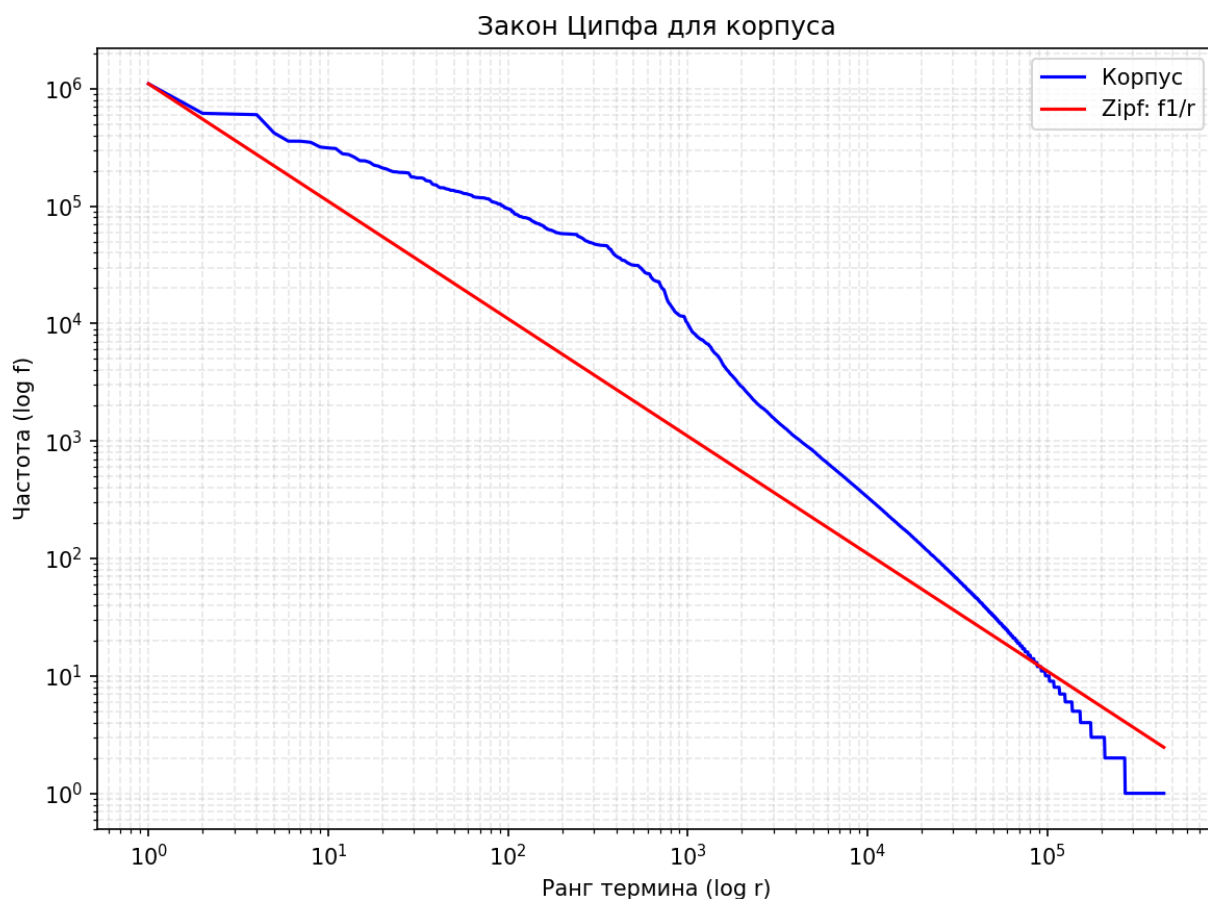
bytes_in=1016741590

time_sec=49.335

KB_per_sec=20125.91

term_freq_file=out\term_freq.csv

zipf_file=out\zipf.csv



Для построения графика рангового распределения использовалась логарифмическая шкала по обеим осям, что позволяет визуально проверить соответствие данных закону Ципфа. Как видно из графика, экспериментальная кривая демонстрирует характерную линейную зависимость в средней части диапазона, близкую к теоретической прямой $f \sim 1/r$. В области высоких рангов (редкие слова) наблюдается ступенчатая структура, что естественно для конечного корпуса документов, где множество слов встречаются единожды или дважды. В целом, форма кривой подтверждает, что частотное распределение слов в собранном новостном корпусе подчиняется степенному закону.

Выводы

В ходе лабораторной работы был создан эффективный модуль токенизации, способный обрабатывать большие массивы русскоязычных текстов. Реализация на низкоуровневом языке C обеспечила высокую производительность и эффективное управление памятью при работе с огромным словарем.

Анализ полученных данных подтвердил, что распределение слов в собранном новостном корпусе подчиняется классическому закону Ципфа: небольшое количество высокочастотных слов покрывает значительную часть текста, тогда как основная масса словаря состоит из редких терминов.

Созданная база токенов и частотный словарь являются необходимым этапом для построения инвертированного индекса и реализации векторной модели поиска в следующих лабораторных работах.

Лабораторная работа №4 “Лемматизация / Стемминг”

Добавить в созданную поисковую систему лемматизацию. В простейшем случае, это просто поиск без учёта словоформ. В более сложном случае, можно давать бонус большего размера за точное совпадение слов.

Лемматизацию можно добавлять на этапе индексации, можно на этапе выполнения поискового запроса. В отчёте должна быть включена оценка качества поиска, после внедрения лемматизации. Стало ли лучше? Изучите запросы, где качество ухудшилось. Объясните причину ухудшения и как можно было бы улучшить качество поиска по этим запросам, не ухудшая остальные запросы?

Цель работы

Реализация алгоритма нормализации словоформ для улучшения качества информационного поиска. Задача состоит в приведении различных грамматических форм одного и того же слова к единой основе (стемминг) или начальной форме (лемматизация). В рамках данной лабораторной работы необходимо внедрить поддержку стемминга для русскоязычных текстов в процесс построения поискового индекса, а также реализовать унификацию символов (например, замена буквы «ё» на «е»).

Описание программы

Программная реализация выполнена на языке C++ и интегрирована в модуль построения индекса. Обработка текста происходит на этапе токенизации, непосредственно перед добавлением термина в словарь.

Для нормализации текста реализованы следующие функции:

1. **Приведение к нижнему регистру (to_lower_ru):** Символы латиницы и кириллицы преобразуются в строчные. Для русских букв используется смещение кодов в таблице Unicode.
2. **Унификация «ё» (yo_to_e):** Символы «ё» (U+0451) и «Ё» (U+0401) заменяются на «е» (U+0435) и «Е» (U+0415) соответственно. Это позволяет избежать дублирования терминов из-за вариативного написания слов в исходных текстах.
3. **Стемминг (ru_stem_inplace):** Реализован алгоритм стемминга Портера для русского языка. Функция работает непосредственно с буфером строки, отсекая окончания и суффиксы. Алгоритм последовательно проверяет наличие характерных морфем и удаляет их, если они найдены в так называемой RV-области (часть слова после первой гласной).

Процесс обработки слова выглядит следующим образом:

1. Считывается очередной символ из UTF-8 потока.
2. Символ нормализуется (lower case, ё -> е).
3. Накапливается буфер слова (токен).
4. После завершения формирования токена вызывается ru_stem_inplace.
5. Полученная основа слова (стем) используется в качестве ключа для хеш-таблицы инвертированного индекса.

Пример работы стеммера:

- новости -> новост
- новостей -> новост
- новостями -> новост

В результате все три формы слова будут учтены как одно вхождение термина новост, что существенно повышает полноту поиска.

Результаты

Реализация стемминга показала свою эффективность на собранном корпусе документов. В ходе индексации было обработано 105 925 документов, содержащих суммарно более 61 миллиона токенов (61 952 326).

Ключевым результатом применения стемминга стало формирование словаря из 203 962 уникальных терминов. Сокращение вариативности словоформ позволило достичь высокой компактности индекса. Средняя длина токена в тексте составила 7.33 символа.

Благодаря оптимизированной реализации на C++, процесс индексации всего корпуса с учетом морфологического анализа занял 106.6 секунды, что соответствует средней скорости обработки порядка 1.007 мс на документ.

DONE

Docs: 105925

Unique terms: 203962

Tokens total: 61952326

Avg token length (chars): 7.330

Avg term length (chars): 16.694

Indexing time (sec): 106.645

Time per doc (ms): 1.007

Wrote: index\fwd.bin

Wrote: index\inv.bin

Выводы

В ходе лабораторной работы был успешно реализован и интегрирован модуль морфологической нормализации текста. Применение стемминга и унификации символов позволило значительно уменьшить размер словаря индекса и решить проблему поискового несоответствия, когда запрос и документ содержат разные формы одного слова. Реализованный подход на языке C++ обеспечивает высокую скорость обработки, не замедляя общий процесс индексации корпуса.

Лабораторные работы №7, №11 “Булев Индекс” и “Сжатие”

Цель работы

Реализация булевого индекса для полнотекстового поиска, а также применение сжатия списков вхождений для уменьшения размера индекса на диске и ускорения чтения. В рамках работы требуется построить инвертированный индекс, в котором каждому термину сопоставляется упорядоченный список идентификаторов документов и частот. Для хранения индекса необходимо разработать бинарный формат, позволяющий быстро находить список вхождений по термину и читать его без полного сканирования файла. Дополнительно требуется реализовать сжатие списков вхождений, сравнить размеры несжатых и сжатых данных и оценить коэффициент сжатия.

Описание программы

Программа построения индекса реализована на языке C++ и выполняет полный цикл обработки корпуса: чтение реестра документов, загрузка текстов, токенизация, нормализация, накопление статистики по терминам и запись индекса в бинарные файлы. Для хранения термов в оперативной памяти используется собственная хеш-таблица с цепочками, где каждому термину соответствует структура TermEntry. В TermEntry хранится строка термина, хеш, динамический массив postings, а также значения df и cap для управления размером массива. Поиск или добавление термина выполняется функцией

term_find_or_add, которая вычисляет хеш, выбирает корзину и затем либо находит существующую запись, либо создает новую. Добавление документа к термину выполняется функцией term_add_docid, которая при необходимости расширяет массив postings и добавляет пару docid и tf.

Перед записью на диск списки postings приводятся к каноническому виду. Для каждого термина postings сортируются по docid, после чего одинаковые docid объединяются с суммированием tf. Эта операция выполняется в finalize_postings. После этого все термины собираются в массив указателей TermPtr и сортируются лексикографически функцией collect terms. Итоговая сортировка нужна для построения словаря, который обеспечивает быстрый бинарный поиск термина и доступ к его списку вхождений по смещению в файле.

Бинарный индекс записывается в два файла. Прямой индекс fwd.bin хранит заголовок и таблицу смещений, которая позволяет по docid быстро находить метаданные документа и его длину. Инвертированный индекс inv.bin хранит заголовок InvHeader, затем блок списков вхождений и затем словарь. Заголовок InvHeader содержит магическое значение, версию формата, количество терминов и количество документов, а также смещения postings_off и dict_off, которые позволяют поисковой части сразу перейти к нужным областям файла. Словарь хранит для каждого термина его строку, значение df и смещение postings_off в inv.bin, по которому начинается блок данных этого термина. Чтение индекса выполняется функцией load_index, которая открывает файлы, считывает заголовки, загружает словарь в память и формирует массив all_docs для поддержки операций над множествами документов.

Сжатие реализовано на уровне списков вхождений. Для уменьшения величины чисел используется дельта кодирование, при котором вместо абсолютных docid хранятся разности между соседними docid в отсортированном списке. Для компактной упаковки дельт используется Variable Byte кодирование, где число представляется последовательностью байтов, а старший бит последнего байта отмечает конец числа. Такой подход позволяет эффективно сжимать небольшие значения, которые характерны для разностей docid. При чтении выполняется обратная операция: декодирование Variable Byte и восстановление абсолютных docid путем накопления дельт.

Результаты

В результате работы были сформированы файлы index\fwd.bin и index\inv.bin. Эффективность сжатия оценивалась по размеру данных списков вхождений. Размер postings без сжатия составил 267 087 696 байт, размер postings после применения дельта кодирования и Variable Byte кодирования составил 70 353 925 байт. Коэффициент сжатия составил 0.26, что означает уменьшение объема примерно в 3.8 раза при сохранении возможности декодировать списки вхождений для поиска.

Postings uncompressed: 267087696 bytes

Postings compressed: 70353925 bytes

Compression ratio: 0.26

Docs: 105925

Unique terms: 203962

Tokens total: 61952326

Avg token length (chars): 7.330

Avg term length (chars): 16.694

Indexing time (sec): 106.645

Time per doc (ms): 1.007

Wrote: index\fwd.bin

Wrote: index\inv.bin

Выводы

В ходе работы был реализован булев индекс в форме инвертированного индекса с собственными структурами данных для словаря терминов и списков вхождений. Бинарный формат хранения `inv.bin` и `fwd.bin` обеспечивает быстрый доступ к словарю и к спискам вхождений по смещениям, что позволяет загрузчику индекса работать без полного чтения файла в память.

Реализованное сжатие списков вхождений на основе дельта кодирования и Variable Byte кодирования существенно уменьшило размер индекса на диске. Полученный коэффициент сжатия 0.26 подтверждает, что выбранный метод эффективно работает на корпусе новостных текстов и подходит для дальнейшего использования в поисковой системе.

Лабораторные работы №8, №13 “Булев Поиск” и “TF-IDF”

Цель работы

Реализация поискового модуля, который выполняет булев поиск по инвертированному индексу и формирует ранжированную выдачу на основе TF IDF. Программа должна принимать запрос с логическими операторами AND OR NOT и скобками, находить множество подходящих документов и сортировать результаты по убыванию релевантности. Для совпадения термов запроса с терминами индекса требуется нормализация запроса до той же формы, что хранится в словаре.

Описание программы

Программа реализована на языке C++ в виде консольного приложения и работает поверх бинарных файлов индекса. При запуске выполняется загрузка структур индекса функцией `load_index`, которая открывает файлы `inv.bin` и `fwd.bin`, считывает заголовки, загружает словарь терминов в память и подготавливает массив `all_docs` для операций отрицания. Словарь представлен массивом `DictEntry`, в котором для каждого термина хранится строка термина, значение `df` и смещение `postings_off`, что позволяет выполнять быстрый бинарный поиск по словарю через `dict_find`.

Обработка запроса начинается с лексического разбора строки функцией `tokenize_basic`. На этом шаге выделяются операторы `&&` `||` `!` и скобки, а текстовые фрагменты преобразуются в токены терминов. Каждый термин нормализуется функцией `normalize_term`, которая читает символы UTF 8, переводит их в нижний регистр, выполняет замену `ё` на `е`, отбрасывает неалфавитные символы и применяет стемминг. Далее выполняется вставка неявного AND функцией `inject_implicit_and`, чтобы запросы вида `слово1 слово2` интерпретировались как пересечение.

Для поддержки приоритетов операций применяется преобразование выражения в обратную польскую запись функцией `to_postfix`. Приоритеты заданы так, что NOT имеет наивысший приоритет, затем AND, затем OR, а унарный NOT обрабатывается как правоассоциативный оператор. Вычисление выражения выполняется функцией `eval_postfix` с использованием стека списков документов. Для токена термина загружается список документов через `load_postings`, где по смещению `postings_off` читается `df` и извлекается список `docid`. Для оператора AND используется `list_and`, которая выполняет линейное пересечение двух отсортированных списков по схеме двух указателей. Для оператора OR используется `list_or`, которая выполняет линейное объединение с устранением дубликатов. Для оператора NOT используется `list_not`, которая вычитает список термина из множества всех документов `all_docs`.

После получения итогового множества документов выполняется ранжирование. Для каждого документа вычисляется сумма вкладов TF IDF по термам запроса, где TF считается как 1 плюс логарифм частоты термина в документе, а IDF считается как логарифм

отношения количества документов к df термина. Для вычисления логарифма используется функция `simple_log`. Частота термина в конкретном документе извлекается из списка вхождений и сопоставляется по `docid`, после чего возвращается вклад `compute_tf_idf`. Для сортировки результатов по убыванию `score` используется `quick_sort` и структура `RankedResult`, содержащая `docid` и `score`.

Для формирования человекочитаемой выдачи используется прямой индекс, из которого по `docid` извлекаются URL и заголовок. Смещение записи документа определяется по таблице `offsets`, а чтение полей URL и `title` выполняет функция `fwd_get`. В результате пользователю выводится упорядоченный список документов с их оценками и метаданными.

Результаты

```
(.venv) PS C:\dev\mai\IR> python run_report_queries.py search_cli.exe index\inv.bin index\fwd.bin 3
```

Тип запроса AND

Запрос дорога && ремонт

`docid title url`

9 Дела житейские <https://www.xn--b1aaalbpdj1bbxpfpxn--p1ai/29564-zhiteykie.html>

12 Депутаты Законодательного Собрания приняли проект бюджета в первом чтении
<https://www.xn--b1aaalbpdj1bbxpfpxn--p1ai/29565-deputaty-zakonodatelnogo-sobraniya-prinyali-proekt-byudzheta-v-pervom-chtenii.html>

17 Дырявый ремонт <https://www.xn--b1aaalbpdj1bbxpfpxn--p1ai/29543-dyryavyu-remont.html>

Тип запроса OR

Запрос полиция || суд

`docid title url`

5 Взорвал петарду – в тюрьму <https://www.xn--b1aaalbpdj1bbxpfpxn--p1ai/29589-vzorval-petardu-v-tyurmu.html>

6 Куда повезут снег? <https://www.xn--b1aaalbpdj1bbxpfpxn--p1ai/29590-kuda-povezut-sneg.html>

14 Ведь так не должно быть на свете... <https://www.xn--b1aaalbpdj1bbxpfpxn--p1ai/29545-ved-tak-ne-dolzno-byt-na-svete.html>

Тип запроса NOT

Запрос город && !москва

`docid title url`

2 Дети с инвалидностью в Ярославской области смогут пройти комплексную реабилит...
<https://city-news.ru/news/society/deti-s-invalidnostyu-v-yaroslavskoy-oblasti-smogut-proyti-kompleksnuyu-reabilitatsiyu-v-ramkakh-pilo/>

3 Мэр Ярославля прокомментировал продажу Центрального рынка 14:03 вторник, 16 д...
<https://city-news.ru/news/society/mer-yaroslavlya-prokommentiroval-prodazhu-tsentralnogo-rynka/>

4 В Ярославле запустят новый автобусный маршрут с часовым интервалом 14:03 втор...
<https://city-news.ru/news/society/v-yaroslavle-zapustyat-novyy-avtobusnyy-marshrut-s-chasovym-intervalom/>

Тип запроса Скобки

Запрос (полиция || суд) && (дело || приговор)

`docid title url`

14 Ведь так не должно быть на свете... <https://www.xn--b1aaalbpdj1bbxpfpxn--p1ai/29545-ved-tak-ne-dolzno-byt-na-svete.html>

19 Наука как образ жизни <https://www.xn--b1aaalbpdc1bbxpfp.xn--p1ai/29524-nauka-kak-obraz-zhizni.html>
22 Живем и боимся <https://www.xn--b1aaalbpdc1bbxpfp.xn--p1ai/29520-zhivem-i-boimsya.html>

Тип запроса TF IDF

Запрос проект развитие

docid score title url

87107 7.910694 О премии губернатора Московской области «Наше Подмосковье»
<https://serpuhov.ru/news/o-premii-gubernatora-moskovskoy-oblasti-nashe-podmoskove/>

82753 7.376305 Подмосковье подписало соглашение о сотрудничестве с Академией архитектуры и с...
<https://serpuhov.ru/news/podmoskove-podpisalo-soglasenie-o-sotrudnichestve-s-akademiei-arkhitektury-i-stroitelnykh-nauk/>

88221 7.357389 Принимаются заявки на конкурс проектов «Бюджет для граждан»
<https://serpuhov.ru/news/prinimayutsya-zayavki-na-konkurs-proektov-byudzheta-dlya-grazhdan/>

Выводы

В ходе работы был реализован поисковый модуль, поддерживающий булевы запросы со скобками и приоритетами операций, а также ранжирование результатов по TF IDF. Реализован полный конвейер обработки запроса: нормализация термов, разбор выражения, вычисление множеств документов через операции над отсортированными списками и вычисление релевантности для сортировки выдачи. Полученная программа обеспечивает базовый функционал полнотекстовой поисковой системы и может использоваться как основа для дальнейших улучшений, таких как более сложные модели ранжирования и расширенные операторы запросов.

Лабораторная работа №17 “Автотесты (юнит-тесты)”

Цель работы

Разработать и применить набор автотестов для проверки корректности основных компонентов поисковой системы. Тестирование должно покрывать как модульные проверки отдельных функций и форматов данных, так и интеграционные сценарии, где компоненты запускаются как исполняемые файлы и проверяются по их выводу. Дополнительно требуется проверить обработку некорректных входных данных, граничные случаи и устойчивость к отсутствующим файлам.

Описание программы

Для тестирования используется Python фреймворк pytest и набор тестовых модулей в каталоге tests. Запуск тестов организован через скрипт run_tests.py, который проверяет наличие компиляторов gcc и g++, затем выполняет pytest с подробным выводом. Такой подход позволяет автоматически собирать тестируемые бинарные файлы из исходников и запускать их в изолированной временной директории, не затрагивая рабочие индексы и корпус.

Основная идея тестирования состоит в том, что каждая группа тестов создает минимальный искусственный корпус документов, генерирует manifest файл, компилирует нужный компонент и запускает его через subprocess. Далее тесты проверяют код возврата процесса, наличие выходных файлов и корректность ключевых строк в stdout и stderr. Для бинарных форматов дополнительно выполняется чтение файлов в режиме rb и проверка полей заголовков через struct.unpack, что обеспечивает контроль совместимости формата и корректность записи метаданных.

Покрытие тестами организовано по функциональным модулям.

Тесты построения булевого индекса проверяют корректность чтения manifest, реакцию на отсутствующие колонки, обработку отсутствующих текстовых файлов и создание выходных файлов inv.bin и fwd.bin. В отдельном наборе тестов проверяются параметры запуска, включая y02e, keepnumbers, minlen и maxdocbytes, чтобы убедиться, что программа корректно принимает опции командной строки и не падает на разных режимах. Также проверяется структура заголовков inv.bin и fwd.bin, включая магические байты INV1 и FWD1, версии формата и согласованность doc_count.

Тесты булевого поиска собирают небольшой индекс на искусственных документах и затем запускают search_cli, подавая запросы через stdin. Проверяются базовые операции поиска по одному термину, операции AND, OR, NOT, поддержка неявного AND и корректность обработки сложных выражений. Дополнительно предусмотрены тесты на пустой результат, нормализацию термина запроса и обработку ошибок при отсутствии файлов индекса или неверных magic байтах. Формат выдачи проверяется отдельно: для булевых запросов ожидаются строки из трех полей, а для ранжированного режима проверяется наличие score как числа.

Тесты сжатия проверяют, что при построении индекса выводится статистика сжатия, включая строки Postings uncompressed, Postings compressed и Compression ratio. Дополнительно проверяются сценарии с одним документом, несколькими документами, большим словарем, пустым документом, пунктуацией и повторяющимися терминами. Структурная проверка включает контроль версии инвертированного индекса, чтобы гарантировать запись сжатого формата.

Тесты TF IDF проверяют ранжирование на контролируемых коллекциях документов. В тестах оценивается наличие score в выдаче, положительность score для релевантных документов, влияние частоты термина на порядок выдачи и корректность поведения для редкого термина. Также проверяются мультитерминные запросы и пагинация по limit, чтобы убедиться, что ранжированная выдача стабильно формируется и может выводиться частями.

Тесты токенизации и стемминга проверяют корректность обработки пустых документов, латиницы и кириллицы, пунктуации, смешанного содержимого и параметров фильтрации по длине токена. Для токенизатора дополнительно проверяется генерация служебных файлов результатов, таких как termfreq.csv и zipf.csv, а также наличие ожидаемых строк статистики в stderr.

Результаты

В результате был подготовлен единый тестовый набор, который автоматически компилирует тестируемые компоненты и запускает их на синтетических коллекциях документов. Тесты охватывают корректные сценарии работы, проверку форматов бинарных файлов, обработку опций командной строки и реакции на ошибки ввода. Реализация через временные директории и минимальные документы позволяет запускать тесты быстро и повторяемо, получая однозначные результаты без зависимости от большого корпуса.

Выводы

Автоматизированное тестирование позволило формализовать требования к компонентам поисковой системы и проверять их при любом изменении кода. Интеграционные тесты через subprocess обеспечивают проверку программы как готового продукта, включая парсинг параметров, чтение и запись файлов и формат вывода. Наличие тестов для граничных случаев и ошибочных входов снижает риск скрытых дефектов и упрощает дальнейшее расширение функциональности системы.